

Proyecto 1
Algoritmos de búsqueda

Presentado por:

Juan Andrés Bravo Mejia
David Alejandro Davila Guaranguay
Arturo Alejandro Manrique Montoya
María José Zabala Acero

Asignatura

Introducción a la inteligencia artificial

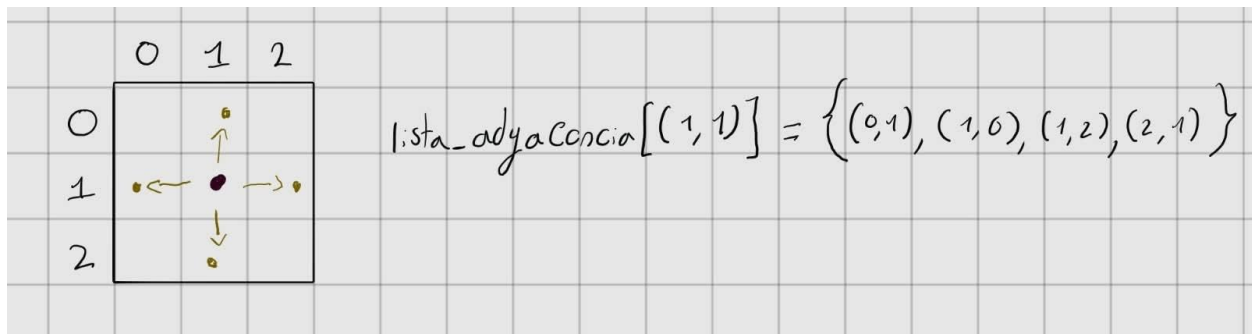
Docente

Julio Omar Ancizar Palacio Niño

PONTIFICIA UNIVERSIDAD JAVERIANA
BOGOTÁ D.C. 2026

Construcción del grafo

Bajo el formato establecido, se definió al laberinto con una cuadrícula que en sus respectivas casillas tiene un número, si este número es 0 es una casilla libre, si el número es un 1 es una pared si el número es un 2 era el punto de inicio para el jugador y si el número es un 3 era el punto de destino. Con base a esto se construyó el grafo tomando como nodos las casillas representadas como una pareja de r y c (row and column) sus aristas se construyen a partir de los 4 movimientos posibles (arriba, abajo, izquierda y derecha) pero si uno de los nodos es conectado con una pared o fuera del rango esta conexión no se efectúa dando como resultado una lista de adyacencia donde por cada nodo tiene sus respectivas conexiones.



De esta manera construimos el grafo y obtenemos como resultado la lista de adyacencia para representarlo computacionalmente.

Algoritmo DFS

El algoritmo de Búsqueda en Profundidad (DFS) fue diseñado con una premisa distinta a la de BFS: en lugar de explorar el grafo de manera uniforme y concéntrica, avanza lo más posible por una misma rama antes de retroceder. Ante un nodo con varios vecinos no visitados, elige uno, se compromete con esa dirección y solo regresa cuando llega a un callejón sin salida, retomando entonces la siguiente alternativa disponible.

Para lograr este comportamiento, el algoritmo utiliza una estructura de datos de tipo LIFO (último en entrar, primero en salir), representada mediante una pila explícita o, de forma equivalente, mediante las llamadas recursivas de la función (donde la pila de ejecución del lenguaje cumple ese rol). Al igual que en BFS, se mantiene un conjunto de nodos visitados para evitar ciclos infinitos y reprocesamiento, verificando y registrando cada nodo en tiempo promedio $O(1)$.

La reconstrucción de la ruta sigue la misma lógica que BFS: se conserva un diccionario de padres donde cada nodo descubierto guarda una referencia al nodo desde el cual fue alcanzado. Una vez encontrado el destino, se reconstruye el camino siguiendo los padres en sentido inverso hasta llegar al inicial y luego invirtiendo la lista resultante para obtener la secuencia correcta.

A diferencia de BFS, DFS no garantiza que el camino encontrado sea el más corto. Al comprometerse con una rama antes de explorar alternativas, puede llegar a la meta por una ruta más larga que la óptima. Su ventaja está en el consumo de memoria: mientras BFS debe mantener en la cola todos los nodos de un mismo nivel, DFS solo necesita recordar la rama actual y sus puntos de retroceso, lo que lo hace más eficiente en grafos muy anchos.

En cuanto a complejidad temporal, se mantiene en $O(V + E)$, igual que BFS, ya que cada nodo se visita a lo sumo una vez y cada arista se evalúa una única vez al expandir sus vecinos.

Este algoritmo representa, junto con BFS, la otra mitad del estándar de búsqueda ciega: no incorpora conocimiento espacial del entorno y, aunque renuncia a la optimalidad en la longitud de la ruta, ofrece una exploración con menor huella de memoria, útil como referencia frente a algoritmos informados como A^* .

Algoritmo BFS

El algoritmo de Búsqueda en Anchura (BFS) fue construido bajo la premisa de que la forma más directa y justa de explorar un grafo no ponderado es expandirse de manera uniforme en todas las direcciones posibles, tratando a cada arista con la misma relevancia. A diferencia de enfoques basados en prioridades o heurísticas, este algoritmo no intenta adivinar qué camino parece más prometedor hacia el destino; en su lugar, garantiza encontrar la ruta más corta en términos de número de pasos explorando sistemáticamente nivel por nivel, asegurando que ningún nodo a una distancia $k + 1$ sea evaluado antes de haber agotado todos los nodos a distancia k .

Para implementar esta exploración concéntrica, el algoritmo se apoya en una estructura de datos de tipo cola FIFO (primero en entrar, primero en salir), representada eficientemente en el código mediante una deque. Esta estructura permite insertar nuevos vecinos al final y extraer elementos por el frente en tiempo constante $O(1)$. Al mismo tiempo, para evitar ciclos infinitos y reprocesar estados innecesarios en laberintos o grafos con mallas cerradas, se utiliza una tabla de dispersión de visitados (un set), la cual permite verificar y registrar en tiempo promedio $O(1)$ si un nodo ya ha sido alcanzado previamente.

Otro aspecto fundamental del algoritmo radica en la reconstrucción de la solución una vez que el nodo actual coincide con el nodo final. Dado que la cola solo gestiona el orden de exploración y descarta los elementos tras expandirlos, el algoritmo mantiene un mapa o diccionario de padres. Cada vez que se descubre un vecino no visitado, se almacena una referencia que apunta directamente hacia el nodo desde el cual fue descubierto. Al alcanzar la meta, basta con realizar un seguimiento inverso desde el nodo final hasta llegar al nodo inicial (donde el padre es None), acumulando los nodos intermedios e invirtiendo la lista resultante para presentar el trayecto en el orden cronológico correcto de avance.

Al no incorporar funciones heurísticas de distancia que sesguen la dirección del avance, la exploración consume una mayor cantidad de nodos en comparación con algoritmos informados, debiendo expandir un radio completo de posibilidades. Sin embargo, su complejidad temporal se mantiene estrictamente acotada en $O(V + E)$, donde V representa la cantidad de vértices explorados y E el número de aristas transitadas, ya que cada nodo ingresa y sale de la cola a lo sumo una única vez y cada conexión se evalúa de manera directa.

Este algoritmo representa el estándar fundamental de búsqueda ciega: prescinde por completo del conocimiento espacial o geométrico del entorno, pero a cambio ofrece una garantía matemática sólida de encontrar siempre el camino más corto en grafos sin pesos, sirviendo como la base conceptual sobre la cual se diseñan técnicas de navegación más avanzadas.

Algoritmo A*

El algoritmo A* fue construido con la idea de que, aunque un nodo tenga cuatro diferentes aristas hay aristas que preferimos recorrer más que otras, cuando llevamos esta idea al recorrido general del grafo, establecemos que podemos crear una escala de prioridad para seleccionar el orden en donde los nodos van a ser recorridos, intentando de esta manera obtener una respuesta casi optima sin tanta necesidad de carga computacional.

Partiendo de esta idea establecemos una heurística para evaluar estos diferentes nodos con base a su efectividad para encontrar un camino al nodo de destino. Cuando intentamos evaluar las diferentes heurísticas que podemos utilizar nos damos cuenta de que una excelente manera de evaluar la efectividad de expandir un nodo es la distancia de este frente a el nodo de destino, la primera idea que puede llegar a surgir es usar la distancia de Euclides la cual establece que la distancia de un punto (x_1, y_1) y otro punto (x_2, y_2) es igual a la raíz cuadrada de las diferencias de sus coordenadas al cuadrado, aunque es una excelente medida de distancia para coordenadas decimales, nuestro problema es mucho más simple que eso ya que como el laberinto es representado como una cuadrícula ninguno de sus puntos se encuentra en una coordenada decimal, teniendo esto en cuenta cuando intentamos calcular la distancia de Euclides correremos el riesgo a tomar cálculos inexactos debido al punto flotante, una mejor estrategia para calcular la distancia entre 2 puntos que no corre el riesgo de pasar por punto flotante es la distancia de Manhattan que es igual al valor absoluto de las diferencias de sus coordenadas, además esta distancia es excelente para evaluar la cantidad de pasos necesarios que toma ir de un punto a otro sin movimiento diagonales, modelando de manera perfecta las condiciones de nuestro laberinto.

Otro análisis necesario para finalizar este algoritmo es que no siempre expandir únicamente por los nodos que son mejor heurísticamente nos garantiza obtener una solución que sea suficientemente eficiente, en este caso una solución que sea lo suficientemente corta. Entonces también debemos de alguna manera castigar al algoritmo si su expansión es demasiado alta, ya que si esto ocurre la solución podría terminar siendo demasiado extensa. Para esto establecemos

una función que nos calcule su costo de expansión, o en este caso el mínimo nivel en el que se encuentra este nodo cuando es recorrido, de esta manera se penaliza al algoritmo si comienza a tomar un camino en donde su expansión es demasiado alta, balanceando la heurística anterior con la expectativa de encontrar una solución suficientemente corta.

Con estas dos funciones que van a evaluar cada nodo a ser expandido, podemos generar un algoritmo que comience en el nodo inicial realice su expansión y busque tomar el nodo con el mínimo costo de expansión tanto heurístico como de nivel, para hacer esto utilizamos una cola de prioridad que organiza la lista de menor a mayor garantizando que el primer nodo de la lista siempre sea el de mínimo costo de nuestra función. Todo esto resultando en una complejidad de $O(V \log(v) + E)$ debido a que máximo cada nodo va a ser agregado una vez a la cola de prioridad que toma $\log(v)$ de complejidad en agregarlo y al menos vamos a recorrer cada arista una vez.

Este algoritmo nos muestra el intento de balancear la toma de decisiones locales sin dejar de lado la búsqueda de una solución óptima global, al balancear estos dos ejes de pensamiento encontramos un algoritmo que no siempre encuentra la solución más óptima posible, pero es mucho más eficiente que los algoritmos estándar que la encuentran.